DRY

Don’t Repeat Yourself Principle

One authoritative source of knowledge

DEFINITION

Every piece of knowledge should have a single, unambiguous representation in the system.

DRY is about knowledge, not characters. The real target is a single decision — a business rule, a formula, a schema — that must change in one place. Two snippets that merely look alike but encode different decisions are not duplication; copying the one rule that has to stay consistent is. The test is whether the copies must change together.

WHY IT MATTERS

When one rule lives in several places, an update fixes some copies and forgets others, and they drift until a bug exposes the inconsistency. A single source means one edit, no divergence, and a clear place to add a test. But the cure has a dark side: deduplicating things that only resemble each other couples them, so DRY is a judgment call, not a reflex.

Vasyl Krupka · Ukraine

How it works

Knowledge	A single fact or decision the system encodes — a tax rate, a validation rule, a status enum. The unit DRY actually cares about.
Single source	The one module that owns that knowledge. Everyone else derives from it (imports the constant, calls the function) rather than restating it.
Derivation	Anything generated from the source instead of hand-copied — types from a schema, clients from an API spec, docs from code.

HOW TO APPLY

1. When you copy-paste, pause: is this the same decision, or just similar-looking code?
2. If it is one decision, name it once — a constant, function, type or module — and point callers at it.
3. Prefer generating derived artifacts (types, clients) from a single definition over parallel copies.
4. If two copies start needing to differ, that is a signal they were never the same knowledge — let them split.

LITMUS TEST

If this rule changed, how many places would I edit? More than one means the knowledge is duplicated. Then ask: must the copies change together, or do they only match today?

Vasyl Krupka · Ukraine

— Smell → fix

The 20% VAT rule is hard-coded in three modules:

```
const price = net + net * 0.2 // checkout
const due = subtotal + subtotal * 0.2 // invoice
const tax = amount * 0.2 // report
// change the rate and you must find every copy
```

↓ EXTRACT THE SINGLE SOURCE OF TRUTH

```
const VAT_RATE = 0.2 // one source of knowledge
function withVat(net) { return net + net * VAT_RATE }
// checkout, invoice and report all call withVat()
```

The rate lives in one constant behind one function. When VAT changes, exactly one line changes — and no caller can drift out of sync.

USE WHEN

- ✓ The same business rule is copy-pasted in three places
- ✓ Magic numbers duplicated everywhere

AVOID WHEN

- ▲ Coupling unrelated code that merely looks similar (false DRY)

Trade-offs & pitfalls

PROS

+ One place to change
+ Fewer divergence bugs

CONS

− The wrong abstraction is worse than duplication
− Premature DRY couples

COMMON PITFALLS

▲ False DRY: merging two rules that only look alike, then bending the shared code with flags when they diverge.
▲ Hoisting code into a shared "utils" or base class for reuse, coupling unrelated callers through it.
▲ Treating DRY as absolute — a little duplication is cheaper than the wrong abstraction (Metz).

WHERE YOU SEE IT

- One source of truth for a status enum, shared by API, DB and UI instead of three literal lists.
- Generating TypeScript types from the OpenAPI or DB schema rather than hand-writing them twice.
- A single validation schema (e.g. Zod) reused on client and server instead of duplicated checks.

SEE ALSO

SoC	put each kind of knowledge in the module that owns that concern
SRP	a single owner per responsibility naturally avoids duplicated rules
KISS	one source is simpler to reason about and to change
OCP	centralized knowledge is easier to extend without scattering edits

Interview & sources

When is duplication better than DRY?

When the similarity is coincidental. Coupling two unrelated rules because they look alike — "the wrong abstraction" — is worse than a little repetition.

How do you tell true from false duplication?

Ask whether the copies must change together for the same reason. If yes, it is knowledge duplication; if they only happen to match, leave them apart.

Does DRY apply beyond code?

Yes — schema, config, docs and build steps too. Generate derived artifacts from one definition instead of maintaining parallel copies.

How does the rule of three apply to DRY?

Don’t abstract on the first repetition. By the third real occurrence you can see the true shared shape, so the abstraction fits instead of being guessed from two samples.

REMEMBER

Every piece of knowledge has one authoritative home in the system.

SOURCES

Andrew Hunt, David Thomas — "The Pragmatic Programmer" (1999): the original DRY definition

Sandi Metz — "The Wrong Abstraction" (2016): prefer duplication over the wrong abstraction